

INTELLISEARCH V2

Cloud-Hybrid Multimodal RAG Platform

Complete Application Development Prompt Guide

Component	Technology
Backend Framework	FastAPI + Python 3.10+
Vector Database	ChromaDB (Local SQLite)
Perception Engine	GPT-4o via GitHub Models
Logic Engine	Llama 3.1 405B via GitHub Models
Desktop Packaging	PyInstaller → .exe
Frontend	React + Next.js + Tailwind CSS
Containerization	Docker + docker-compose.prod.yml
CI/CD	GitHub Actions + pytest

Master Overview & Project Vision

IntelliSearch V2 is a Cloud-Hybrid Multimodal Retrieval-Augmented Generation (RAG) platform. It acts as a highly intelligent librarian that can see images, listen to audio, and read thousands of pages of text — storing all knowledge in a local ChromaDB vector database and offloading cognitive reasoning to massive cloud models (GPT-4o + Llama 3.1 405B) via the GitHub Models free tier.

The final deliverable is a packaged Windows .exe desktop application that users can launch without any Python or Node.js installation. This guide provides every prompt — in structured phases — to generate the complete codebase from scratch.

Core Architecture: The Dual-Brain Strategy

The system uses two separate GitHub Personal Access Tokens routed through <https://models.github.ai/inference> using the OpenAI Python SDK:

- **GPT-4o (Perception Engine / Token A): Multimodal ingestion** — converts images, audio, and complex PDFs into rich semantic text for ChromaDB.
- **Llama 3.1 405B (Logic Engine / Token B): 128k context window reasoning, cross-document synthesis, and deep citation generation.**

Strategic Advantages

- Privacy: Vector embeddings stay on local machine / private server
- Cost-Efficiency: GitHub Models free tier with dual-token strategy — near-zero API cost
- Trust: Deep Citation feature proves exactly which file each answer came from
- Anti-Hallucination: Similarity Gate (cosine ≥ 0.70) blocks answers when context is insufficient
- Scale: Cloud intelligence on any hardware — no local GPU required

Complete Tech Stack

Layer	Technology	Purpose
API Server	FastAPI + Uvicorn	Async REST endpoints
AI Inference	openai SDK	GitHub Models routing
Vector Store	ChromaDB	Local persistent embeddings
File Processing	PyMuPDF, Pillow, pydub	PDF/image/audio handling
Security	Custom middleware	X-IntelliSearch-Client-Key
Process Mgmt	psutil + subprocess	Ghost process cleanup
Frontend	React + Next.js + Tailwind	SPA chat interface
Animations	Framer Motion	Smooth UI transitions
Packaging	PyInstaller	Windows .exe build
Container	Docker + Compose	Production deployment

Layer	Technology	Purpose
Testing	pytest + httpx	Automated test suite
CI/CD	GitHub Actions	Deploy on push

PHASE 1

Project Scaffold & Environment Setup

Establish the complete folder structure, virtual environment, dependencies, and configuration files before writing a single line of application logic.

GOAL A clean, reproducible project skeleton that any developer can clone and run in under 5 minutes.

Prompt 1.1 — Project Folder Structure

Use the following exact prompt with your AI coding assistant (Claude, Cursor, Copilot):

```

"""
Create the complete folder structure for IntelliSearch V2, a Cloud-Hybrid
Multimodal RAG desktop application. The structure must be:

intellisearch-v2/
├── backend/
│   ├── app/
│   │   ├── __init__.py
│   │   ├── main.py          # FastAPI app entry point
│   │   ├── config.py       # All env vars & constants
│   │   ├── security.py     # Client key middleware
│   │   ├── models.py       # Pydantic request/response models
│   │   ├── routers/
│   │   │   ├── ingest.py   # POST /ingest
│   │   │   └── query.py    # POST /query
│   │   └── services/
│   │       ├── chromadb_service.py
│   │       ├── gpt4o_service.py
│   │       └── llama_service.py
│   └── frontend/
│       ├── app/
│       │   ├── layout.tsx
│       │   └── page.tsx
│       ├── components/
│       │   ├── Sidebar.tsx
│       │   ├── ChatCanvas.tsx
│       │   ├── UploadZone.tsx
│       │   ├── MessageBubble.tsx
│       │   ├── CitationPill.tsx
│       │   ├── TypingIndicator.tsx
│       │   ├── SettingsModal.tsx
│       │   └── ToastNotification.tsx
│       ├── hooks/
│       └── useChat.ts

```

```

|   |   └─ useHealth.ts
|   └─ lib/
|       └─ api.ts           # All fetch calls
├─ chroma_storage/         # Auto-created, gitignored
├─ tests/
|   └─ test_ingest.py
|       └─ test_query.py
├─ .github/workflows/
|   └─ ci.yml
├─ run.py                  # psutil process launcher
├─ build_exe.py            # PyInstaller build script
├─ requirements.txt
├─ docker-compose.prod.yml
├─ Dockerfile
└─ .env.example

```

Generate every `__init__.py` and placeholder file with correct imports.
 Add a `.gitignore` that excludes: `.env`, `chroma_storage/`, `__pycache__/`,
`.next/`, `node_modules/`, `dist/`, `*.exe`, `*.spec`

"""

Prompt 1.2 — requirements.txt & Dependencies

"""

Generate the complete requirements.txt for IntelliSearch V2 backend.
 Include exact pinned versions for:

Core: fastapi==0.111.0, uvicorn[standard]==0.30.1
 AI SDK: openai==1.35.0 (for GitHub Models routing)
 Vector DB: chromadb==0.5.0
 File Processing: PyMuPDF==1.24.5, Pillow==10.3.0, pydub==0.25.1
 Process Mgmt: psutil==5.9.8
 Testing: pytest==8.2.2, httpx==0.27.0, pytest-asyncio==0.23.7
 Packaging: pyinstaller==6.8.0
 Utilities: python-multipart==0.0.9, python-dotenv==1.0.1, aiofiles==23.2.1

Also generate package.json for the Next.js frontend with:
 next@14.2.4, react@18.3.1, react-dom@18.3.1,
 tailwindcss@3.4.4, framer-motion@11.2.12,
 lucide-react@0.383.0, react-markdown@9.0.1,
 remark-gfm@4.0.0, @types/node, @types/react, typescript

"""

Prompt 1.3 — Environment Configuration

"""

Generate the following configuration files for IntelliSearch V2:

1. backend/app/config.py – A Pydantic BaseSettings class called Settings with:
 - GITHUB_TOKEN_A: str (for GPT-4o)
 - GITHUB_TOKEN_B: str (for Llama 3.1)
 - GITHUB_MODELS_BASE_URL: str = "https://models.github.ai/inference"
 - CHROMA_PERSIST_DIR: str = "./chroma_storage"
 - CHROMA_COLLECTION_NAME: str = "intellisearch_v2_corpus"
 - CLIENT_KEY: str (secret key for X-IntelliSearch-Client-Key)
 - GPT4O_MODEL: str = "gpt-4o"
 - LLAMA_MODEL: str = "Meta-Llama-3.1-405B-Instruct"
 - TOP_K_RETRIEVAL: int = 15
 - SIMILARITY_THRESHOLD: float = 0.70
 - CHUNK_SIZE: int = 512
 - CHUNK_OVERLAP: int = 64
 - Load from .env file using model_config = SettingsConfigDict(env_file=".env")
2. .env.example – template with all keys and placeholder values
3. backend/app/models.py – Pydantic models:
 - IngestResponse(status, filename, chunks_stored, file_type)
 - Citation(source, type, chunk_id, similarity_score)
 - QueryRequest(query: str)
 - QueryResponse(answer, citations: List[Citation], top_similarity, gate_passed, model_used)
 - HealthResponse(status, chromadb_connected, corpus_size, models)

PHASE 2

Backend Core — Security & FastAPI Foundation

Build the FastAPI application core: security middleware, CORS, global exception handlers, health endpoint, and the process orchestration launcher.

GOAL A running FastAPI server at `http://127.0.0.1:8000` with authentication, resilience, and observability baked in.

Prompt 2.1 — FastAPI Main Application

```
"""
Generate backend/app/main.py for IntelliSearch V2 FastAPI application.

Requirements:
- Import and create FastAPI app with title="IntelliSearch V2", version="2.0.0"
- Add CORSMiddleware allowing: all origins (*), methods (GET, POST, OPTIONS),
  headers (*, X-IntelliSearch-Client-Key), and credentials=False
- Include routers: ingest_router (prefix="/ingest"), query_router (prefix="/query")
- GET /health endpoint returning HealthResponse: check ChromaDB connection,
  return corpus document count, list active model names
- Global exception handlers:
  * openai.RateLimitError → HTTP 429 JSON {"error": "rate_limit", "message": ...}
  * openai.AuthenticationError → HTTP 401 JSON {"error": "auth_failed", ...}
  * Generic Exception → HTTP 500 JSON {"error": "internal_error", ...}
- Python logging with format: "%(asctime)s [%(levelname)s] %(name)s: %(message)s"
- Log inference start/end with latency in ms and payload size in bytes
- On startup: log "IntelliSearch V2 online" and verify ChromaDB connection
"""
```

Prompt 2.2 — Security Middleware

```
"""
Generate backend/app/security.py for IntelliSearch V2.

Create a FastAPI dependency function verify_client_key(request: Request):
- Extract X-IntelliSearch-Client-Key header from request
- Compare against settings.CLIENT_KEY using secrets.compare_digest()
  for timing-attack-safe comparison
- If header is missing: raise HTTPException(403, "Client key required")
- If header is invalid: raise HTTPException(403, "Invalid client key")
- If valid: return the key string
- Add to /ingest and /query routes as Depends(verify_client_key)
- Do NOT require key for GET /health (monitoring must work unauthenticated)
"""
```

Prompt 2.3 — Process Launcher (run.py)

```
"""
Generate run.py – the standalone process orchestration launcher.

Requirements:
- Import psutil, subprocess, sys, time, logging, socket
- Function kill_ghost_processes(port=8000):
  * Iterate psutil.process_iter() finding processes with connections on port 8000
  * Gracefully terminate (SIGTERM), wait 2s, force kill (SIGKILL) if still alive
  * Log each killed process PID and name
- Function wait_for_server(host, port, timeout=15) → bool:
  * Poll socket.connect_ex every 0.5s until timeout
  * Return True when port accepts connections
- main():
  * Call kill_ghost_processes()
  * Launch uvicorn via subprocess: ["uvicorn", "backend.app.main:app",
    "--host", "127.0.0.1", "--port", "8000", "--reload"]
  * Wait for server using wait_for_server()
  * Print "✓ IntelliSearch V2 is online at http://127.0.0.1:8000"
  * Block on process.wait()
- Handle KeyboardInterrupt gracefully (SIGTERM to uvicorn subprocess)
"""
```


PHASE 3

ChromaDB Vector Database Service

Build the local persistent vector store — the "memory" of the system. This service handles chunking, embedding, storage, and similarity-gated retrieval.

GOAL A complete ChromaDB service that chunks text into 512-token pieces, stores with full metadata, and retrieves the top-15 most similar chunks for any query.

Prompt 3.1 — ChromaDB Service

```
"""
Generate backend/app/services/chromadb_service.py for IntelliSearch V2.

Class: ChromaDBService

__init__:
- Initialize chromadb.PersistentClient(path=settings.CHROMA_PERSIST_DIR)
- Get or create collection: settings.CHROMA_COLLECTION_NAME
- Use default embedding function (all-MiniLM-L6-v2 via chromadb.utils)

chunk_text(text: str, chunk_size: int, overlap: int) → List[str]:
- Split text into chunks of chunk_size characters with overlap
- Do not split mid-word: find nearest whitespace boundary
- Return list of clean chunk strings, minimum length 50 chars

store_chunks(chunks: List[str], source: str, file_type: str) → int:
- Generate chunk IDs: f"{source}_chunk_{i}" for each chunk
- Metadata per chunk: {"source": source, "type": file_type,
  "chunk_index": i, "timestamp": ISO datetime string}
- Add in batches of 50 to avoid memory issues
- Return count of chunks stored
- If source already exists, delete old chunks first (deduplication)

query_corpus(query: str, top_k: int) → dict:
- collection.query(query_texts=[query], n_results=top_k,
  include=["documents", "metadatas", "distances"])
- Convert ChromaDB distance to cosine similarity: sim = 1 - (distance / 2)
- Return: {"chunks": [...], "metadatas": [...], "similarities": [...],
  "top_similarity": float}

get_corpus_size() → int:
- Return collection.count()

delete_source(source: str):
- Delete all chunks where metadata source == source
```

```
Export as singleton: chroma_service = ChromaDBService()  
"""
```

PHASE 4

GPT-4o Perception Engine (Ingestion Service)

Build the multimodal file processing pipeline. GPT-4o acts as the "eyes and ears" — converting every file format into rich semantic text for ChromaDB storage.

GOAL A single ingest endpoint that accepts PDFs, images, and audio — extracts semantic text via GPT-4o — and stores everything in ChromaDB with deep metadata.

Prompt 4.1 — GPT-4o Perception Service

```
"""
Generate backend/app/services/gpt4o_service.py for IntelliSearch V2.

Use: openai.AsyncOpenAI(api_key=settings.GITHUB_TOKEN_A,
                        base_url=settings.GITHUB_MODELS_BASE_URL)

Class: GPT4oService

async extract_from_image(image_bytes: bytes, filename: str) → str:
    - Encode image_bytes as base64
    - Detect MIME type from filename extension (png/jpg/jpeg/webp)
    - Send to GPT-4o with vision: messages=[{role: user, content: [
        {type: image_url, image_url: {url: f"data:{mime};base64,{b64}"}}],
        {type: text, text: VISION_PROMPT}}]]
    - VISION_PROMPT: "You are a semantic extraction engine. Analyze this image
        comprehensively. Describe: all text visible, spatial relationships,
        entities, labels, data in charts/tables, architectural components,
        colors and their meaning, and any technical notation. Be exhaustive.
        Output only dense, structured prose – no preamble."
    - Return response text

async extract_from_audio(audio_bytes: bytes, filename: str) → str:
    - Save audio_bytes to temp file using tempfile.NamedTemporaryFile
    - Use openai.audio.transcriptions.create(model="whisper-1", file=...)
    - After transcription, send transcript to GPT-4o for semantic enrichment:
        "Enrich this transcript with: speaker intent, key decisions, action items,
        named entities, and thematic structure. Format for maximum retrievability."
    - Return enriched text. Clean temp file in finally block.

async extract_from_pdf(pdf_bytes: bytes, filename: str) → str:
    - Use PyMuPDF (fitz): open from bytes, extract text page by page
    - For pages with < 100 chars (scanned/image PDFs): render page as image,
        call extract_from_image() on each such page
    - Combine all page texts with page number markers: "--- Page N ---"
    - If total text > 8000 chars: send to GPT-4o for semantic summarization

```

```

- Return final combined/summarized text
- Handle encrypted PDFs: return error string, do not crash
"""

```

Prompt 4.2 — Ingest Router

```

"""
Generate backend/app/routers/ingest.py for IntelliSearch V2.

POST /ingest endpoint:
- Accept: UploadFile = File(...)
- Dependency: Depends(verify_client_key)
- Returns: IngestResponse

Logic:
1. Read file bytes: content = await file.read()
2. Log: filename, size in KB, mime type
3. Detect file type by extension:
   - .pdf → gpt4o_service.extract_from_pdf()
   - .png/.jpg/.jpeg/.webp → gpt4o_service.extract_from_image()
   - .mp3/.wav/.ogg/.m4a → gpt4o_service.extract_from_audio()
   - other → raise HTTPException(415, "Unsupported file type")
4. Chunk extracted text using chroma_service.chunk_text(
   text, settings.CHUNK_SIZE, settings.CHUNK_OVERLAP)
5. Store: n = chroma_service.store_chunks(chunks, file.filename, file_type)
6. Log total latency and chunk count
7. Return IngestResponse(status="success", filename=file.filename,
   chunks_stored=n, file_type=file_type)

Error handling:
- Wrap in try/except, return IngestResponse(status="error", ...) on failure
- Never let exceptions propagate unhandled to client
"""

```

PHASE 5

Llama 3.1 405B Logic Engine (Query Service)

Build the reasoning pipeline: ChromaDB retrieval → Similarity Gate → Llama synthesis → deep citation mapping. This is the intelligence core.

CRITICAL The Similarity Gate (cosine ≥ 0.70) is the anti-hallucination firewall. If it blocks, the LLM is never called — return the fixed "Information not found" response immediately.

Prompt 5.1 — Llama 3.1 Service

```
"""
Generate backend/app/services/llama_service.py for IntelliSearch V2.

Use: openai.AsyncOpenAI(api_key=settings.GITHUB_TOKEN_B,
                        base_url=settings.GITHUB_MODELS_BASE_URL)
Model: settings.LLAMA_MODEL = "Meta-Llama-3.1-405B-Instruct"

Class: LlamaService

SYSTEM_PROMPT = """
You are IntelliSearch V2, a precise knowledge retrieval system.
You ONLY answer using the provided context chunks.
Rules:
1. Base every claim on the retrieved context — never invent facts.
2. When you use information from a source, you MUST cite it inline
   using [Source: filename] notation.
3. If the context does not contain the answer, say exactly:
   "The information is not available in the ingested documents."
4. Structure long answers with markdown headers and bullet points.
5. Be concise but complete. Prioritize high-similarity chunks.
"""

async synthesizer(query: str, context_chunks: List[str],
                  metadatas: List[dict]) → str:
    - Build context string: for each chunk, prepend
      f"[Source: {meta[source]} | Type: {meta[type]}]\n{chunk}\n---"
    - USER_PROMPT: f"Context:\n{context_str}\n\nQuestion: {query}"
    - Call Llama with: model, max_tokens=4096, temperature=0.1,
      messages=[{system: SYSTEM_PROMPT}, {user: USER_PROMPT}]
    - Log: query length, context length, response length, latency
    - Return response.choices[0].message.content
"""
```

Prompt 5.2 — Query Router with Similarity Gate

```

"""
Generate backend/app/routers/query.py for IntelliSearch V2.

POST /query endpoint:
- Accept: QueryRequest (Pydantic body)
- Dependency: Depends(verify_client_key)
- Returns: QueryResponse

Logic (STRICTLY in this order):

STEP 1 – Validate input:
- Raise 400 if query is empty or > 2000 chars

STEP 2 – ChromaDB Retrieval:
- results = await chroma_service.query_corpus(query, top_k=settings.TOP_K_RETRIEVAL)
- If corpus is empty: return QueryResponse with gate_passed=False,
  answer="No documents have been ingested yet. Please upload files first."

STEP 3 – THE SIMILARITY GATE (anti-hallucination firewall):
- top_sim = results["top_similarity"]
- If top_sim < settings.SIMILARITY_THRESHOLD (0.70):
  * Log: f"Gate BLOCKED: top similarity {top_sim:.3f} < 0.70"
  * Return QueryResponse(
    answer="Information not found: No sufficiently relevant content
    was found in the knowledge base for your query. The closest match
    had a similarity score of {top_sim:.2f} (threshold: 0.70). Try
    rephrasing or ingest documents related to this topic.",
    citations=[], gate_passed=False, top_similarity=top_sim,
    model_used="gate_blocked")

STEP 4 – Llama Synthesis (only if gate passed):
- answer = await llama_service.synthesize(query, results["chunks"],
                                           results["metadatas"])

STEP 5 – Citation Extraction:
- Build citations from metadatas: deduplicate by source filename
- Each Citation: source=meta["source"], type=meta["type"],
  chunk_id=meta["chunk_index"], similarity_score=sim
- Sort citations by similarity_score descending

STEP 6 – Return:
- QueryResponse(answer, citations, top_similarity, gate_passed=True,
  model_used=settings.LLAMA_MODEL)
"""

```

PHASE 6

Next.js Frontend — Complete SPA

Build the complete React/Next.js single-page application with dark mode, glassmorphism, animated components, multimodal upload, chat interface, and deep citation rendering.

GOAL A production-grade Vercel-style dark SPA that communicates with the FastAPI backend, shows real-time status, renders markdown beautifully, and displays deep citation pills.

Prompt 6.1 — Global Layout & Theme

```
"""
Generate the Next.js App Router layout for IntelliSearch V2 frontend.

Files to generate:

1. frontend/app/layout.tsx:
  - Import Google Fonts: Sora (weights 300,400,500,600) and DM Mono (300,400,500)
  - Apply className to html: "dark antialiased"
  - Set metadata: title="IntelliSearch V2", description="Multimodal RAG Platform"
  - Body: bg-zinc-950, text-zinc-100, font-sans, h-screen overflow-hidden

2. tailwind.config.ts:
  - Dark mode: "class"
  - Extend theme with custom colors:
    gold: { DEFAULT: "#C9A84C", light: "#E8C87A", soft: "#F5E0A0" }
    accent: { DEFAULT: "#6366F1", glow: "rgba(99,102,241,0.2)" }
  - Custom fontFamily: sans: ["Sora"], mono: ["DM Mono"]
  - Content: ["/app/**/*.tsx", "/components/**/*.tsx"]

3. frontend/lib/api.ts:
  - BASE_URL = "http://127.0.0.1:8000"
  - getHeaders(clientKey?: string): always include Content-Type,
    add X-IntelliSearch-Client-Key if clientKey provided
  - async checkHealth(): GET /health → HealthResponse
  - async ingestFile(file: File, clientKey: string): POST /ingest
    multipart/form-data → IngestResponse
  - async queryCorpus(query: string, clientKey: string): POST /query
    JSON body → QueryResponse
  - Each function: catches network errors, returns typed response or throws
  - Export TypeScript interfaces matching all Pydantic response models
"""
```

Prompt 6.2 — Sidebar Component

```
"""
```

Generate frontend/components/Sidebar.tsx for IntelliSearch V2.

```
Props: { onNewChat: () => void, onOpenSettings: () => void,
        clientKey: string, ingestedFiles: FileInfo[] }
```

Structure:

- └ Brand header: IS mark + "IntelliSearch" + "RAG V2" pill
- └ New Chat button (accent colored)
- └ "Recent Sessions" label + scrollable list (mocked data: 5 items)
- └ Ingested Files section: list with color-coded file type dots
 - | (red=pdf, green=image, purple=audio)
- └ Footer:
 - └ Health status card (uses useHealth hook, polls every 15s)
 - | - Shows: colored dot, latency in ms, GPT-4o + Llama model tags
 - | - If offline: show "python run.py" instructions
 - └ Configure button with key indicator dot

Styling: bg-zinc-900, border-r border-zinc-800, w-64 flex-shrink-0

Use Framer Motion: sidebar slides in from left on mount (x: -20 → 0)

"""

Prompt 6.3 — Upload Zone Component

Generate frontend/components/UploadZone.tsx for IntelliSearch V2.

```
Props: { clientKey: string, onIngested: (file: FileInfo) => void,
        onToast: (msg: string, type: ToastType) => void }
```

States: idle | dragover | uploading | success | error

Features:

- Drag-and-drop zone with onDragOver/onDrop/onDragLeave handlers
- Hidden <input type="file"> with accept=".pdf,.png,.jpg,.jpeg,.mp3,.wav"
- Client-side validation: reject unsupported extensions before uploading
- During upload:
 - * Animated progress bar (simulated 5%→85%, then jump to 100% on success)
 - * Text: "GPT-4o is analyzing [filename]..."
 - * Orange pulsing dot (GPT-4o brand color)
- On 403 → toast "Invalid Client Key"
- On 429 → toast "GPT-4o rate limit reached – try again shortly"
- On success → toast "✓ [filename] ingested into ChromaDB"
- Framer Motion: scale animation on dragover (1.0 → 1.01)
- Show ingested file list as colored pills below the zone

"""

Prompt 6.4 — Chat Canvas & Message Components


```

"""
Generate frontend/components/ChatCanvas.tsx and MessageBubble.tsx
for IntelliSearch V2.

ChatCanvas:
- Props: { messages: Message[], isTyping: boolean,
  onPromptChip: (text: string) => void }
- If messages.length === 0: render EmptyState component with:
  * Large brain emoji orb with gradient border
  * Title: "Dual-Brain Intelligence" with gold gradient on "Dual-Brain"
  * Subtitle explaining GPT-4o + Llama 3.1 405B
  * 4 clickable capability cards in a 2x2 grid:
    - "Analyze PDFs" / "Vision Analysis" / "Audio Intelligence"
      / "Cross-doc Synthesis"
  * Framer Motion: staggered card reveal (0.1s delay each)
- Messages list: auto-scroll to bottom on new message
- Show TypingIndicator when isTyping=true
- Ref scroll container, useEffect to scroll on messages change

MessageBubble:
- Props: { message: Message }
- User bubble: right-aligned, indigo/accent background
- AI bubble: left-aligned, zinc-900 background, border zinc-800
- AI bubbles show engine chips: "ChromaDB top-15" + "Llama 3.1 405B"
- Gate-blocked messages: amber warning style, shield icon
- Error messages: rose/red style, warning icon
- Render AI content with react-markdown + remark-gfm
  (handles bold, lists, code blocks, tables, blockquotes)
- Citation pills: below AI bubble, color-coded by type
  (red=pdf, green=image, purple=audio)
- Similarity bar: mini progress bar showing cosine score
- Framer Motion: slide up + fade on message entry (y: 10 → 0)
"""

```

Prompt 6.5 — Settings Modal & Chat Input

```

"""
Generate frontend/components/SettingsModal.tsx and the chat input
section for IntelliSearch V2 frontend/app/page.tsx.

SettingsModal:
- Backdrop: fixed inset-0 bg-black/70 backdrop-blur-md
- Card: centered, max-w-md, bg-zinc-900, border zinc-800, rounded-2xl
- Framer Motion: scale 0.95→1.0 + opacity 0→1 on open
- Fields:
  1. Client Key: password input, placeholder "sk-intellisearch-..."
     hint: "Sent as X-IntelliSearch-Client-Key header"
  2. Backend Endpoint: text input, default "http://127.0.0.1:8000"
- Architecture info section:

```

```
* GPT-4o card (orange): "Perception Engine – Token A"
* Llama 3.1 405B card (purple): "Logic Engine – 128k context – Token B"
* Note: "Similarity Gate threshold: 0.70 cosine similarity"
- Buttons: Cancel (secondary) + Save Changes (gold/accent)
- Close on backdrop click or Escape key

Chat Input Section (in page.tsx):
- Auto-resizing textarea (min 1 row, max 6 rows via JS)
- Send on Enter (Shift+Enter = newline)
- Send button: disabled when empty or isTyping
- Footer row:
  Left: green dot + "Llama 3.1 405B · top_k=15 · gate≥0.70"
  Right: char count or "Enter to send"
- Focus ring: indigo glow on focus-within
"""
```

PHASE 7

State Management & React Hooks

Build the custom hooks that manage application state — chat history, health polling, file tracking — keeping components clean and logic centralized.

Prompt 7.1 — useChat Hook

```

"""
Generate frontend/hooks/useChat.ts for IntelliSearch V2.

Custom hook managing all chat state and API interactions:

State:
- messages: Message[] (id, role, content, type, citations?,
  similarity?, time)
- isTyping: boolean
- clientKey: string (persisted to localStorage)
- ingestedFiles: FileInfo[] (name, type)
- toasts: Toast[]

Functions:
sendMessage(query: string):
  1. Append user message immediately
  2. Set isTyping = true
  3. Call api.queryCorpus(query, clientKey)
  4. On success:
    - If gate_passed=false: append gate-blocked AI message
    - If gate_passed=true: append normal AI message with citations
  5. On 403: append error message "Invalid client key"
  6. On 429: append error message + show toast
  7. On 401: append error message "GitHub token invalid"
  8. On network error: show fallback demo message (dev mode)
  9. Always: set isTyping = false

clearChat(): reset messages to []
addToast(msg, type): push toast, auto-remove after 4000ms
setClientKey(key): update state + persist to localStorage

Export: { messages, isTyping, clientKey, ingestedFiles, toasts,
  sendMessage, clearChat, addToast, setClientKey }
"""

```

Prompt 7.2 — useHealth Hook

```

"""
Generate frontend/hooks/useHealth.ts for IntelliSearch V2.

```

Custom hook for backend health monitoring:

State:

- status: "checking" | "online" | "offline"
- latencyMs: number | null
- corpusSize: number
- models: { gpt4o: boolean, llama: boolean }

Logic:

- On mount: immediately call checkHealth()
- setInterval(checkHealth, 15000) – poll every 15 seconds
- checkHealth():
 - * Record t0 = Date.now()
 - * Call api.checkHealth() with 3000ms timeout (AbortController)
 - * On success: set status="online", latencyMs=Date.now()-t0, corpusSize from response
 - * On error: set status="offline", latencyMs=null
- Cleanup: clearInterval on unmount
- Export: { status, latencyMs, corpusSize, models }

""

PHASE 8

Testing Suite

Write the pytest test suite covering all API endpoints, the similarity gate, error handling, and file ingestion. Must be CI/CD ready.

Prompt 8.1 — Test Suite

```
"""
Generate the complete pytest test suite for IntelliSearch V2.

tests/test_ingest.py:
- Use httpx.AsyncClient with app from main.py
- Fixture: valid_client_key – reads from os.environ["CLIENT_KEY"]
- test_ingest_pdf_success: upload sample PDF → expect 200, status="success"
- test_ingest_image_success: upload sample PNG → expect 200
- test_ingest_unsupported_type: upload .txt → expect 415
- test_ingest_missing_client_key: no header → expect 403
- test_ingest_invalid_client_key: wrong key → expect 403
- test_ingest_deduplication: ingest same file twice →
  second ingest replaces first, corpus size stays constant

tests/test_query.py:
- test_query_empty_corpus: query before ingest → gate_passed=False
- test_query_gate_blocked: mock low similarity → gate_passed=False,
  answer contains "Information not found"
- test_query_gate_passes: mock high similarity → gate_passed=True,
  answer is non-empty string
- test_query_returns_citations: citations list not empty when gate passes
- test_query_missing_key: no header → 403
- test_query_empty_string: empty query → 400
- test_query_too_long: 2001 char query → 400
- test_health_endpoint: GET /health → 200, status field present

tests/conftest.py:
- pytest.ini settings: asyncio_mode = "auto"
- Fixtures for sample PDF bytes and sample PNG bytes (minimal valid files)
- Override settings.SIMILARITY_THRESHOLD to 0.0 for integration tests
"""
```

PHASE 9

Docker & Production Infrastructure

Build the complete Docker containerization, GitHub Actions CI/CD pipeline, and production deployment configuration.

Prompt 9.1 — Dockerfile & Docker Compose

```

"""
Generate Dockerfile and docker-compose.prod.yml for IntelliSearch V2.

Dockerfile:
FROM python:3.10-slim
- Set WORKDIR /app
- Install system deps: ffmpeg (for audio), libgl1 (for PyMuPDF)
- COPY requirements.txt → pip install --no-cache-dir -r requirements.txt
- COPY backend/ ./backend/
- COPY .env .env (if exists, else use ENV vars)
- Create volume mount point: /app/chroma_storage
- EXPOSE 8000
- CMD ["uvicorn", "backend.app.main:app", "--host", "0.0.0.0",
      "--port", "8000", "--workers", "2"]

docker-compose.prod.yml:
services:
  intellisearch-backend:
    build: .
    ports: ["8000:8000"]
    volumes: ["/chroma_storage:/app/chroma_storage"]
    env_file: [".env"]
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
      interval: 30s, timeout: 10s, retries: 3
"""

```

Prompt 9.2 — GitHub Actions CI/CD

```

"""
Generate .github/workflows/ci.yml for IntelliSearch V2.

Trigger: push to main and pull_request to main

Jobs:

1. test-backend:
  runs-on: ubuntu-latest

```

```
steps:
- uses: actions/checkout@v4
- Set up Python 3.10
- pip install -r requirements.txt
- Set env vars: CLIENT_KEY=test-key,
  GITHUB_TOKEN_A=dummy, GITHUB_TOKEN_B=dummy
- Run: pytest tests/ -v --tb=short --asyncio-mode=auto
- Upload coverage report as artifact

2. lint-backend:
- ruff check backend/ (fast Python linter)
- mypy backend/ --ignore-missing-imports

3. test-frontend:
- npm install in frontend/
- npx tsc --noEmit (TypeScript type check)
- npm run build (ensure no build errors)

4. docker-build (on main only):
- Build Docker image: intellisearch-v2:latest
- Push to GitHub Container Registry (ghcr.io)
"""
```

PHASE 10

Desktop .EXE Packaging (PyInstaller)

Package the entire application — FastAPI backend + bundled frontend build — into a single Windows .exe file that users can run without any Python installation.

CRITICAL The .exe bundles: Uvicorn server, ChromaDB, all AI SDK dependencies, and serves the pre-built Next.js frontend as static files. Users get ONE double-click launcher.

Prompt 10.1 — PyInstaller Build Script

```
"""
Generate build_exe.py – the complete PyInstaller packaging script
for IntelliSearch V2 Windows .exe build.

Strategy:
  The .exe will:
  1. Start a Uvicorn server (FastAPI backend) in a background thread
  2. Serve the Next.js exported static build via FastAPI StaticFiles
  3. Open the default browser to http://127.0.0.1:8000
  4. Show a system tray icon with a "Quit" option

Pre-build steps in build_exe.py:
  1. cd frontend && npm run build && npm run export
     (generates frontend/out/ static HTML)
  2. Run PyInstaller with:
     --name "IntelliSearch-V2"
     --onefile
     --windowed (no console on Windows)
     --icon assets/icon.ico
     --add-data "frontend/out;frontend/out"
     --add-data "backend;backend"
     --hidden-import chromadb
     --hidden-import chromadb.utils.embedding_functions
     --hidden-import uvicorn.logging
     --hidden-import uvicorn.loops
     --hidden-import uvicorn.loops.auto
     --collect-all chromadb
     --collect-all tiktoken
     entry.py

Generate entry.py (the .exe entry point):
  - Get base path: sys._MEIPASS if frozen, else os.getcwd()
  - Mount frontend/out as StaticFiles at "/" in FastAPI
  - Start uvicorn in daemon Thread
  - Wait for port 8000 to be ready (poll socket)
```


- webbrowser.open("http://127.0.0.1:8000")
- Show pystray system tray icon with Quit option
- Block on tray.run()

Also generate:

- intellisearch.spec fallback spec file
- .github/workflows/build-exe.yml that builds on Windows runner and uploads IntelliSearch-V2.exe as GitHub Release artifact

"""

Prompt 10.2 — EXE Setup & First-Run Experience

"""

Generate the first-run setup experience for the IntelliSearch V2 .exe.

When the .exe is launched for the first time:

1. Check if .env exists next to the .exe
2. If not: show a setup window using tkinter (always available in PyInstaller):
 - Title: "IntelliSearch V2 – First Run Setup"
 - Fields:
 - * GitHub Token A (GPT-4o) – password field
 - * GitHub Token B (Llama 3.1) – password field
 - * Client Key – text field with auto-generate button
 - Instructions link: "How to get GitHub Models tokens"
 - Save button: writes .env file next to .exe
3. After .env exists: proceed to launch backend + open browser

Generate setup_window.py using tkinter:

- Dark theme: bg=#0B0C12, fg=#F5F3EE, accent=#C9A84C
- Auto-generate CLIENT_KEY: secrets.token_hex(32)
- Validate tokens are non-empty before saving
- Show success message and auto-close after 2s

"""

PHASE 11

Integration, Debugging & Final Polish

Connect all layers, run the full stack, fix common integration issues, and polish the application for a production-ready release.

Prompt 11.1 — Full Integration Test

```
"""
Generate an integration test script: scripts/integration_test.py

This script performs an end-to-end test of the live running system:

Prerequisites: Backend must be running at http://127.0.0.1:8000

Tests (in order):
1. Health check: GET /health → assert status="ok"
2. Auth test: POST /ingest without key → assert 403
3. Ingest PDF: create a synthetic 2-page PDF with PyMuPDF, upload it
  → assert IngestResponse.status="success", chunks_stored > 0
4. Ingest image: create a synthetic PNG with Pillow (white image + text)
  → assert success
5. Query – gate blocked: ask "What is the capital of Mars?"
  → assert gate_passed=False
6. Query – gate passes: ask content from the synthetic PDF
  → assert gate_passed=True, citations not empty
7. Citation check: verify citation sources match uploaded filenames
8. Health check: corpus_size should be > 0 now

Print a colored summary table of pass/fail results.
Exit with code 0 if all pass, 1 if any fail.
"""
```

Prompt 11.2 — Common Issues & Fixes

```
"""
Generate a troubleshooting guide and auto-fix script:
scripts/doctor.py

Checks to perform:
1. Python version >= 3.10 (sys.version_info)
2. All requirements.txt packages importable
3. .env file exists and has required keys
4. GitHub Token A is valid (test models.github.ai/inference ping)
5. GitHub Token B is valid
6. Port 8000 is free (not already in use)
7. ChromaDB storage directory writable
```

```

8. ffmpeg installed (for audio ingestion)
9. Node.js >= 18 installed (for frontend build)
10. frontend/out/ exists (static build present for .exe mode)

```

For each check: print ✓ PASS or ✗ FAIL with specific fix instructions.

At the end: print overall health score X/10.

If any FAIL: print the single most impactful fix command to run.

```

"""

```

Prompt 11.3 — README & Documentation

```

"""

```

Generate a complete README.md for the IntelliSearch V2 repository.

Sections:

1. Hero: project name, one-line description, badges (Python, FastAPI, ChromaDB, License)
2. What it does: 3-sentence plain English explanation
3. Architecture diagram: ASCII art showing the dual-brain flow
4. Quick Start (4 steps):
 - a. Clone repo
 - b. `cp .env.example .env` && fill in tokens
 - c. `pip install -r requirements.txt`
 - d. `python run.py`
5. Using the .exe:
 - a. Download IntelliSearch-V2.exe from GitHub Releases
 - b. Double-click → first-run setup wizard appears
 - c. Enter GitHub tokens → app launches in browser
6. API Reference: table of endpoints, methods, auth, body, response
7. Development: how to run tests (pytest), how to run frontend (npm run dev)
8. How to get GitHub Models tokens (step-by-step with screenshots placeholders)
9. Architecture deep-dive: Dual-Brain, Similarity Gate, ChromaDB
10. License: MIT

```

"""

```

PHASE 12

EXE Build & Distribution

Final build pipeline to generate the distributable Windows .exe, create a GitHub Release, and prepare white-label documentation.

Build Sequence — Run in Order

Execute the following commands in sequence to produce the final .exe:

1. Activate virtual environment:

```
python -m venv .venv && .venv\Scripts\activate (Windows)
```

2. Install all dependencies:

```
pip install -r requirements.txt
```

3. Run the full test suite:

```
pytest tests/ -v --asyncio-mode=auto
```

4. Build the Next.js frontend:

```
cd frontend && npm install && npm run build && npm run export && cd ..
```

5. Run the doctor script:

```
python scripts/doctor.py
```

6. Build the .exe:

```
python build_exe.py
```

7. Test the .exe:

```
dist\IntelliSearch-V2.exe
```

8. Tag and release:

```
git tag v2.0.0 && git push origin v2.0.0
```

GitHub Actions EXE Build (Automated)

```
"""
```

Generate .github/workflows/build-exe.yml:

Trigger: push of tag matching v*.*.*

Job: build-windows-exe

```
runs-on: windows-latest
```

```
steps:
```

- actions/checkout@v4
- Set up Python 3.10
- Set up Node.js 20
- pip install -r requirements.txt
- cd frontend && npm ci && npm run build && npm run export
- python build_exe.py
- Upload artifact: dist/IntelliSearch-V2.exe

```
- Create GitHub Release using softprops/action-gh-release@v2
  with the .exe as release asset
"""
```

White-Label Checklist

To sell IntelliSearch V2 as a custom product for a client:

- Replace "IntelliSearch" with client brand name in: config.py, layout.tsx, README.md
- Swap icon.ico and brand colors in tailwind.config.ts
- Update CHROMA_COLLECTION_NAME to client-specific namespace
- Set CLIENT_KEY to a production-strength secret (secrets.token_hex(64))
- Change SIMILARITY_THRESHOLD based on client corpus quality (0.65–0.80 range)
- Build and deliver: dist/ClientName-Intelligence.exe + setup guide

Appendix — Quick Reference

API Endpoints Summary

Method	Endpoint	Auth	Description
GET	/health	None	Backend status, corpus size, model info
POST	/ingest	Required	Upload PDF/image/audio → ChromaDB
POST	/query	Required	Ask question → gated RAG → cited answer

Key Configuration Values

Setting	Default	Notes
SIMILARITY_THRESHOLD	0.70	Cosine score minimum for LLM call
TOP_K_RETRIEVAL	15	ChromaDB chunks returned per query
CHUNK_SIZE	512	Characters per vector chunk
CHUNK_OVERLAP	64	Overlap between adjacent chunks
LLAMA_MODEL	Meta-Llama-3.1-405B-Instruct	GitHub Models name
GPT4O_MODEL	gpt-4o	GitHub Models name

File Type Support Matrix

Extension	Engine	Method	ChromaDB chunks
.pdf	GPT-4o	Text extract + vision fallback	Text split at 512 chars
.png / .jpg	GPT-4o Vision	Base64 image → text description	Single rich chunk
.mp3 / .wav	Whisper + GPT-4o	Transcribe + semantic enrich	Multiple chunks
.jpeg / .webp	GPT-4o Vision	Base64 image → text description	Single rich chunk

IntelliSearch V2 — Complete Development Prompt Guide

Built for MakersGroup · GPT-4o · Llama 3.1 405B · ChromaDB · FastAPI · PyInstaller